

Learning the Graph Structure for Hierarchical Navigable Small Worlds (HNSW) in Approximate Nearest Neighbor Search

Jaskirat Singh Maskeen
IIT Gandhinagar
Gandhinagar, Gujarat, India
23110146@iitgn.ac.in

Ridham Patel
IIT Gandhinagar
Gandhinagar, Gujarat, India
23110238@iitgn.ac.in

Aarsh Wankar
IIT Gandhinagar
Gandhinagar, Gujarat, India
23110003@iitgn.ac.in

Nishchay Bhutoria
IIT Gandhinagar
Gandhinagar, Gujarat, India
23110222@iitgn.ac.in

ABSTRACT

This project aims to explore the feasibility of learning the graph structure used in Hierarchical Navigable Small Worlds (HNSW), a state-of-the-art algorithm for approximate nearest neighbor search. By leveraging machine learning techniques, we seek to optimize the construction of the HNSW graph, potentially enhancing its efficiency and accuracy in high-dimensional spaces. The project will involve analyzing existing HNSW implementations, developing a novel approach to learn the graph structure from given query examples and their outputs, and evaluating the performance of the learned structure against traditional HNSW methods.

KEYWORDS

Approximate Nearest Neighbor Search, Hierarchical Navigable Small Worlds, Graph Learning, High-Dimensional Data

1 INTRODUCTION

Approximate Nearest Neighbor (ANN) search is a crucial task in various applications, including data retrieval and machine learning. Among the ANN algorithms, Hierarchical Navigable Small Worlds (HNSW) stands out for its efficiency and accuracy in high-dimensional spaces. HNSW organizes data into a multi-layered graph structure, facilitating fast and robust searches. However, the construction of this graph is typically based on heuristics rather than learned from data. This project proposes to explore the possibility of learning the graph structure for HNSW using machine learning techniques, potentially leading to improved performance.

2 RELATED WORK

HNSW is a graph-based ANN search algorithm introduced in [6] that builds upon the concept of Navigable Small Worlds (NSW) (introduced in [5]) by introducing a hierarchical structure. This structure allows for efficient navigation across the dataset by skipping irrelevant data portions. Traditional HNSW graph construction relies on parameters like the number of layers and the edge pruning heuristic to balance search efficiency and accuracy. Recent studies have focused on optimizing these parameters but have not explored learning the graph structure directly from data. In the paper [7], the authors introduce ranking mechanisms using ML techniques to prune low-quality neighbours while searching for the nearest nodes to a query by using ML based techniques. However, even

here, the focus is on efficient traversal by reducing the number of distance computations undertaken, and not on learning a good search graph itself.

3 METHODOLOGY

We have three novel machine learning approaches to replace heuristic-based graph construction in HNSW, building upon recent advancements in learned index structures and graph neural networks. On the top of which we also propose some search optimization methods for HNSW as mentioned in 3.4.

3.1 Learnable Random Graph Construction

Drawing inspiration from random graph theory that underpins small-world networks, our first approach begins with a fully connected graph where each edge is assigned a random normalized weight between 0 and 1. Similar to the Erdős-Rényi model but with adaptive weights, we initially prune connections with weights below 0.5 to maintain navigability. When processing a query, the system traces a path through these probabilistic connections to find the nearest neighbor. For each query where the traced path leads to an incorrect node, we calculate the distance error between the predicted and actual target. This error is then distributed proportionally across all edges in the traversed path, reducing their weights to discourage similar incorrect routes in future queries. Over successive iterations, this feedback mechanism allows the graph to organically evolve its connectivity patterns while preserving the small-world properties essential for efficient navigation [4].

3.2 Dynamic Edge Weight Prediction

Our second approach introduces query-aware dynamic edge weighting using Multi-Layer Perceptrons (MLPs). Instead of static graph connections, we employ an MLP to predict edge existence probabilities for each query dynamically. The MLP takes as input the concatenated features of two nodes and outputs a scalar weight representing the edge's relevance to the current query. This allows the graph structure to adapt on-the-fly to different query contexts. During training, we optimize the MLP using a triplet loss function, encouraging it to assign higher weights to edges leading to correct nearest neighbors while penalizing irrelevant connections. By maintaining separate MLP instances for each layer of the HNSW

hierarchy, we preserve the multi-scale nature of the index while enabling flexible, learned connectivity patterns. This method aims to strike a balance between the efficiency of pre-computed graph structures and the adaptability of query-time computations, potentially leading to more accurate and efficient nearest neighbor search.

3.3 Hierarchical Graph Neural Architecture

Our third approach introduces hierarchical message passing within the GNN framework to learn optimal graph connectivity. Unlike conventional GNNs that operate on flat graphs, we design a multi-scale architecture where message propagation occurs across three levels of the HNSW hierarchy:

- (1) Between layers to preserve vertical navigation shortcuts,
- (2) Within layers to refine local neighborhood connections, and
- (3) Across distant clusters to enable long-range hops.

During training, we process input queries through this hierarchical graph network, accumulating node embeddings through successive message passing stages. The final predicted nearest neighbor is compared against the ground truth target node using a contrastive loss function. This loss measures both the correctness of the selected neighbor and the efficiency of the search path. We then backpropagate gradients through the entire message passing hierarchy to simultaneously optimize:

- (1) The edge connection weights at each layer,
- (2) The attention mechanisms governing inter-layer transitions, and
- (3) The priority scoring functions for path selection.

This end-to-end differentiable approach allows the graph structure to automatically adapt to the data distribution and query patterns, potentially overcoming the limitations of handcrafted heuristics in traditional HNSW construction [2].

3.4 Search Optimization Techniques for HNSW

Our methodology enhances HNSW search through four complementary strategies. First, adaptive early termination combines distance-based heuristics with ML predictions: searches stop when current results satisfy $\max_{v \in R_k} \langle q, v \rangle - \epsilon \geq \min_{u \in C} \langle q, u \rangle$, where R_k represents found neighbors and C the candidate set. A neural network simultaneously predicts optimal stopping points using query patterns $\{\delta_i\}_{i=1}^t$ to balance accuracy-speed tradeoffs [1, 8]. Second, our hybrid filtering dynamically switches between pre-filtering (applying constraints first when selectivity exceeds 65%) and in-filtering (embedding filters during traversal for selective queries below 35% selectivity), guided by a cost optimizer that evaluates $\text{FilterRatio} = \frac{\text{FilteredRows}}{\text{TotalRows}}$.

Third, we automate parameter tuning through Bayesian optimization, searching the space of critical HNSW parameters: connection count M (16-48), construction quality $efConstruction$ (100-400), search accuracy $efSearch$ (50-200), and hierarchy balance mL (0.62-0.85). This optimization maximizes $\theta^* =_{\theta} \mathbb{E}[R(\theta)] - \lambda \cdot \text{Cost}(\theta)$, balancing recall against latency/memory costs [1, 3]. Finally, our hybrid architecture combines HNSW with IVF coarse quantization

(implemented via FAISS' IndexIVF_HNSW) and 4-bit product quantization, reducing memory footprint to $\frac{n \cdot d}{8} \cdot \text{bits} + \text{GraphOverhead}$ while maintaining search accuracy through multi-stage filtering [3, 8].

4 DATASETS AND LIBRARIES

Our project utilizes standard benchmarks like SIFT1M and GloVe-100M, alongside synthetic data for stress testing. Key libraries include PyTorch for ML model implementation, Faiss for baseline HNSW, and HNSWlib for custom modifications. Hardware requirements are substantial, with memory needs scaling to hundreds of GBs depending on dataset size, and a minimum of 8 CPU cores recommended. Optional GPU acceleration with NVIDIA T4 or A100 can enhance training efficiency. Optimization strategies such as 4-bit product quantization and IVF-HNSW hybrids help manage resource demands.

5 TIMELINE

- **Week 1 (Mar 27 - Apr 2):** Setup development environment, literature review on HNSW variants, and baseline implementation
- **Week 2 (Apr 3 - 9):** Implement learnable random graph construction with error feedback (Idea 1)
- **Week 3 (Apr 10 - 16):** Develop dynamic edge weighting MLP system (Idea 2) and begin GNN architecture (Idea 3)
- **Week 4 (Apr 17 - 23):** Finalize hierarchical GNN implementation, conduct comparative evaluation, and prepare final report

REFERENCES

- [1] Dmitry Baranchuk, Nikolay Malkin, Yulia Rubanova, and Dmitry Vetrov. 2019. Learning to stop while learning to predict. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, Vol. 97. PMLR, 450–460.
- [2] Vijay Prakash Dwivedi and Xavier Bresson. 2018. Benchmarking Graph Neural Networks. *arXiv preprint arXiv:1812.08434* (2018).
- [3] Liangliang Fan, Qing Lin, Qiang Zhang, Xueyan Yang, Yiqiu Li, and Man Lung Yiu. 2020. Fast approximate nearest neighbor search with the navigating spreading-out graph. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. ACM, 919–934.
- [4] Alan Frieze and Michał Karoński. 2016. *Introduction to Random Graphs*. Cambridge University Press. <https://www.math.cmu.edu/~af1p/BOOK.pdf>
- [5] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45 (2014), 61–68.
- [6] Yu. A. Malkov and D. A. Yashunin. 2018. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *arXiv:1603.09320 [cs.DS]* <https://arxiv.org/abs/1603.09320>
- [7] Yun Peng, Byron Choi, Tsz Nam Chan, and Jianliang Xu. 2022. Lan: Learning-based approximate k-nearest neighbor search in graph databases. In *2022 IEEE 38th international conference on data engineering (ICDE)*. IEEE, 2508–2521.
- [8] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. 2020. CatBoost: unbiased boosting with categorical features. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, Vol. 119. PMLR, 5591–5600.